



Министерство науки и высшего образования Российской Федерации
Федеральное автономное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8
по курсу «Методы оптимизации»
«Реализация метода Пауэла»

Студент группы ИУ9-81Б Кириленко У.О.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовать метод Пауэла.

2 Результаты

Исходный код программы представлен в листингах 3– 3.

Листинг 1: Основное задание

```
1 using LinearAlgebra
2 using Statistics
3 using Plots
4
5 plotly()
6
7 function rosenbrock(x)
8     return (1.0 - x[1])^2 + 100.0 * (x[2] - x[1]^2)^2
9 end
10
11 function rastrigin(x)
12     return 20.0 + x[1]^2 - 10.0*cos(2*pi*x[1]) + x[2]^2 - 10.0*cos(2*pi*
13         x[2])
14 end
15 function schwefel(x)
16     return 418.9829*2 - (x[1]*sin(sqrt(abs(x[1])))) + x[2]*sin(sqrt(abs(x
17         [2])))
18 end
19 struct TestFunction
20     func::Function
21     name::String
22     start_point::Vector{Float64}
23     bounds::Vector{Float64}
24     initial_step::Float64
25 end
26
27 mutable struct SimplexHistory
28     simplices::Vector{Vector{Vector{Float64}}}}
29 end
30
31 function nelder_mead(tf::TestFunction; eps=1e-5, max_iter=50)
32     alpha_param, beta_param, gamma_param, delta_param = 1.0, 0.5, 2.0,
33         0.5
```

```

33     n = length(tf.start_point)
34     history = SimplexHistory([])
35
36     simplex = [copy(tf.start_point) for _ in 1:(n+1)]
37     for i in 1:n
38         simplex[i+1][i] += tf.initial_step
39     end
40     push!(history.simplices, deepcopy(simplex))
41
42     f_vals = [tf.func(v) for v in simplex]
43
44     iter = 0
45     while iter < max_iter
46         iter += 1
47         perm = sortperm(f_vals)
48         simplex = simplex[perm]
49         f_vals = f_vals[perm]
50
51         if std(f_vals) < eps break end
52
53         best, worst = simplex[1], simplex[end]
54         f_best, f_worst = f_vals[1], f_vals[end]
55         f_second_worst = f_vals[end-1]
56
57         centroid = sum(simplex[1:end-1]) / n
58
59         xr = centroid + alpha_param * (centroid - worst)
60         fr = tf.func(xr)
61
62         if f_best <= fr < f_second_worst
63             simplex[end] = xr; f_vals[end] = fr
64         elseif fr < f_best
65             xe = centroid + gamma_param * (xr - centroid)
66             fe = tf.func(xe)
67             if fe < fr
68                 simplex[end] = xe; f_vals[end] = fe
69             else
70                 simplex[end] = xr; f_vals[end] = fr
71             end
72         else
73             if fr < f_worst
74                 xc = centroid + beta_param * (xr - centroid)
75                 fc = tf.func(xc)
76                 if fc <= fr
77                     simplex[end] = xc; f_vals[end] = fc
78                 else

```

```

79             for i in 2:(n+1)
80                 simplex[i] = best + delta_param * (simplex[i] -
best)
81                 f_vals[i] = tf.func(simplex[i])
82             end
83         end
84     else
85         xc = centroid + beta_param * (worst - centroid)
86         fc = tf.func(xc)
87         if fc < f_worst
88             simplex[end] = xc; f_vals[end] = fc
89         else
90             for i in 2:(n+1)
91                 simplex[i] = best + delta_param * (simplex[i] -
best)
92                 f_vals[i] = tf.func(simplex[i])
93             end
94         end
95     end
96 end
97 push!(history.simplices, deepcopy(simplex))
98 end
99 return simplex[1], iter, history
100 end
101
102 function powell_method(f, x0; max_iter=200, eps=1e-4)
103     x = copy(x0)
104     history = [copy(x)]
105     n = length(x)
106
107     # 1:
108     D = [zeros(n) for _ in 1:n]
109     for i in 1:n D[i][i] = 1.0 end
110
111     for iter in 1:max_iter
112         P = copy(x)
113
114         # 2:
115
116         for i in 1:n
117             d = D[i]
118
119             #
120             alpha
121             inner_f(alpha) = f(P + alpha[1] * d)
122             tf = TestFunction(inner_f, "search", [0.0], [], 0.5)

```

```

121
122         #
123         best_alpha = newton_1d(inner_f, 0.0)
124         # best_alpha, _, _ = nelder_mead(tf, max_iter=30)
125
126         P = P + best_alpha[1] * d
127         push!(history, copy(P))
128     end
129
130     #           3:
131
132     d_new = P - x
133
134     if norm(d_new) < eps break end
135
136     #           4:
137
138     inner_f_new(alpha) = f(P + alpha[1] * d_new)
139     tf_new = TestFunction(inner_f_new, "search", [0.0], [], 0.5)
140
141     # best_alpha_new, _, _ = nelder_mead(tf_new, max_iter=30)
142     best_alpha_new = newton_1d(inner_f_new, 0.0)
143
144     x_new = P + best_alpha_new[1] * d_new
145     push!(history, copy(x_new))
146
147     #
148     for i in 1:(n-1) D[i] = D[i+1] end
149     D[n] = d_new / norm(d_new)
150
151     #           :
152
153     D_matrix = hcat(D...)
154     #
155
156     if abs(det(D_matrix)) < 1e-3
157         for i in 1:n
158             D[i] = zeros(n)
159             D[i][i] = 1.0
160         end
161     end
162
163     x = x_new
164 end
165
166 return x, history

```

```

163 end
164
165 function visualize_powell(f, name, x0, bounds_pad)
166     println("          : $name |          : $x0")
167
168     x_res, hist = powell_method(f, x0)
169     println("          :          = $(length(hist
170 )-1),          = $(round.(x_res, digits=3))
171 ")
172
173     all_x = [p[1] for p in hist]
174     all_y = [p[2] for p in hist]
175
176     xr = range(min(minimum(all_x)-bounds_pad, x0[1]-bounds_pad), max(
177 maximum(all_x)+bounds_pad, x0[1]+bounds_pad), length=100)
178     yr = range(min(minimum(all_y)-bounds_pad, x0[2]-bounds_pad), max(
179 maximum(all_y)+bounds_pad, x0[2]+bounds_pad), length=100)
180
181     Z = [f([x, y]) for y in yr, x in xr]
182
183     #
184     p1 = contour(xr, yr, Z, levels=50, color=:grays, fill=false, title="
185 $name (2D)", xlabel="x1", ylabel="x2")
186     plot!(p1, all_x, all_y, color=:purple, lw=2, marker=:circle,
187 markersize=3, label="")
188     scatter!(p1, [x0[1]], [x0[2]], color=:green, markersize=6, label="
189 ")
190     scatter!(p1, [x_res[1]], [x_res[2]], color=:gold, markersize=6,
191 marker=:diamond, label="")
192
193     # 3D
194     p2 = surface(xr, yr, Z, color=:grays, alpha=0.5, colorbar=false,
195 title="3D", xlabel="x1", ylabel="x2")
196     plot!(p2, all_x, all_y, [f(p) for p in hist], color=:purple, lw=3,
197 label="")
198     scatter!(p2, [x0[1]], [x0[2]], [f(x0)], color=:green, markersize=5,
199 label="")
200     scatter!(p2, [x_res[1]], [x_res[2]], [f(x_res)], color=:gold,
201 markersize=5, marker=:diamond, label="")
202
203     display(plot(p1, p2, layout=(1,2), size=(1100, 500)))
204 end

```

Листинг 2: Отрисовка d

```

1 function powell_method(f, x0; max_iter=200, eps=1e-4)
2     x = copy(x0)

```

```

3     history = [copy(x)]
4
5     dirs_history = []
6     n = length(x)
7
8     #         1:
9     D = [zeros(n) for _ in 1:n]
10    for i in 1:n D[i][i] = 1.0 end
11
12    for iter in 1:max_iter
13        P = copy(x)
14
15        #         2:
16
17        for i in 1:n
18            push!(dirs_history, (copy(P), deepcopy(D)))
19
20            d = D[i]
21            inner_f(alpha) = f(P + alpha[1] * d)
22            tf = TestFunction(inner_f, "search", [0.0], [], 0.5)
23
24            best_alpha = newton_1d(inner_f, 0.0)
25            # best_alpha, _, _ = nelder_mead(tf, max_iter=30)
26
27            P = P + best_alpha[1] * d
28            push!(history, copy(P))
29        end
30
31        #         3:
32        d_new = P - x
33        if norm(d_new) < eps break end
34
35        current_basis = deepcopy(D)
36        push!(current_basis, copy(d_new))
37        push!(dirs_history, (copy(P), current_basis))
38
39        #         4:
40
41        inner_f_new(alpha) = f(P + alpha[1] * d_new)
42        tf_new = TestFunction(inner_f_new, "search", [0.0], [], 0.5)
43
44        # best_alpha_new, _, _ = nelder_mead(tf_new, max_iter=30)
45        best_alpha_new = newton_1d(inner_f_new, 0.0)
46
47        x_new = P + best_alpha_new[1] * d_new
48        push!(history, copy(x_new))

```

```

47
48     #
49     for i in 1:(n-1) D[i] = D[i+1] end
50     D[n] = d_new / norm(d_new)
51
52     D_matrix = heat(D...)
53     if abs(det(D_matrix)) < 1e-3
54         for i in 1:n
55             D[i] = zeros(n)
56             D[i][i] = 1.0
57         end
58     end
59
60     x = x_new
61 end
62
63 return x, history, dirs_history
64 end
65
66 function visualize_powell(f, name, x0, bounds_pad)
67     println("          : $name |          : $x0")
68
69     x_res, hist, dirs_hist = powell_method(f, x0)
70     println("          :          =
71     $(length(hist) 3),          = $(round.(
72     x_res, digits=3))")
73
74
75     all_x = [p[1] for p in hist]
76     all_y = [p[2] for p in hist]
77
78     xr = range(min(minimum(all_x)-bounds_pad, x0[1]-bounds_pad), max(
79     maximum(all_x)+bounds_pad, x0[1]+bounds_pad), length=100)
80     yr = range(min(minimum(all_y)-bounds_pad, x0[2]-bounds_pad), max(
81     maximum(all_y)+bounds_pad, x0[2]+bounds_pad), length=100)
82
83     Z = [f([x, y]) for y in yr, x in xr]
84
85     orig_x = Float64[]
86     orig_y = Float64[]
87     dir_u = Float64[]
88     dir_v = Float64[]
89
90     arrow_len = (maximum(xr) - minimum(xr)) * 0.10
91
92     for (P, dirs) in dirs_hist
93         for d in dirs

```

```

89         if norm(d) > 1e-8
90             d_norm = (d / norm(d)) * arrow_len
91             push!(orig_x, P[1])
92             push!(orig_y, P[2])
93             push!(dir_u, d_norm[1])
94             push!(dir_v, d_norm[2])
95         end
96     end
97 end
98
99 p1 = contour(xr, yr, Z, levels=50, color=:grays, fill=false, title="$name (2D)", xlabel="x1", ylabel="x2")
100
101 #
102 plot!(p1, all_x, all_y, color=:purple, lw=2, marker=:circle,
103        markersize=3, label="")
104
105 #
106 quiver!(p1, orig_x, orig_y, quiver=(dir_u, dir_v), color=:blue,
107         alpha=0.6, lw=1.5)
108
109 scatter!(p1, [x0[1]], [x0[2]], color=:green, markersize=6, label="")
110
111 scatter!(p1, [x_res[1]], [x_res[2]], color=:gold, markersize=6,
112         marker=:diamond, label="")
113
114 # 3D
115 p2 = surface(xr, yr, Z, color=:grays, alpha=0.5, colorbar=false,
116             title="3D", xlabel="x1", ylabel="x2")
117 plot!(p2, all_x, all_y, [f(p) for p in hist], color=:purple, lw=3,
118        label="")
119 scatter!(p2, [x0[1]], [x0[2]], [f(x0)], color=:green, markersize=5,
120         label="")
121 scatter!(p2, [x_res[1]], [x_res[2]], [f(x_res)], color=:gold,
122         markersize=5, marker=:diamond, label="")
123
124 display(plot(p1, p2, layout=(1,2), size=(1100, 500)))
125 end
126
127 visualize_powell(rosenbrock, " ", [-1.2, 1.0], 0.5)

```

Листинг 3: LU разложение и метод Ньютона

```

1 function newton_1d_simplified(g, alpha0=0.0; tol=1e-5, max_iter=30, h=1e
2     -5)
3     alpha = alpha0

```

```

4      #                                     ("
          ")
5      d2f = (g(alpha0 + h) - 2 * g(alpha0) + g(alpha0 - h)) / (h^2)
6      if abs(d2f) < 1e-8
7          return alpha0
8      end
9
10     for iter in 1:max_iter
11         df = (g(alpha + h) - g(alpha - h)) / (2 * h)
12
13         step = df / d2f
14         alpha = alpha - step
15
16         if abs(step) < tol || abs(df) < tol
17             break
18         end
19     end
20
21     return alpha
22 end
23
24 function newton_1d(g, alpha0=0.0; tol=1e-5, max_iter=30, h=1e-5)
25     alpha = alpha0
26
27     for iter in 1:max_iter
28         df = (g(alpha + h) - g(alpha - h)) / (2 * h)
29         d2f = (g(alpha + h) - 2 * g(alpha) + g(alpha - h)) / (h^2)
30
31         if abs(d2f) < 1e-8
32             break
33         end
34
35         step = df / d2f
36         alpha = alpha - step
37
38         if abs(step) < tol || abs(df) < tol
39             break
40         end
41     end
42
43     return alpha
44 end
45
46 function manual_lu(A)
47     n = size(A, 1)
48     L = zeros(Float64, n, n)

```

```

49     U = zeros(Float64, n, n)
50
51     for i in 1:n
52         U[i, i] = 1.0
53     end
54
55     for j in 1:n
56         for i in j:n
57             sum_val = 0.0
58             for s in 1:(j-1)
59                 sum_val += L[i, s] * U[s, j]
60             end
61             L[i, j] = A[i, j] - sum_val
62         end
63
64         for k in (j+1):n
65             sum_val = 0.0
66             for s in 1:(j-1)
67                 sum_val += L[j, s] * U[s, k]
68             end
69
70             if abs(L[j, j]) < 1e-12
71                 U[j, k] = 0.0
72             else
73                 U[j, k] = (A[j, k] - sum_val) / L[j, j]
74             end
75         end
76     end
77     return L, U
78 end
79
80 function manual_det(A)
81     n = size(A, 1)
82     L, U = manual_lu(A)
83
84     det_val = 1.0
85     for i in 1:n
86         det_val *= L[i, i]
87     end
88
89     return det_val
90 end

```

Результат запуска представлен на рисунке 1– 3.

ФУНКЦИЯ: Розенброк | Старт: [-1.2, 1.0]
Метод Пауэлла: Итераций = 35, Найденный минимум = [1.0, 1.0]

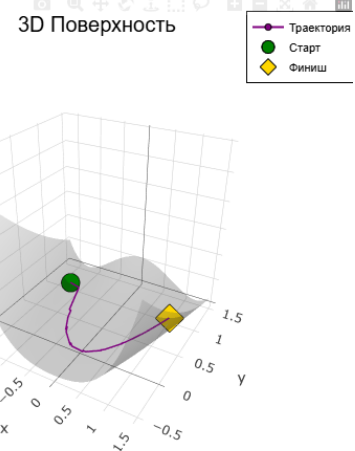
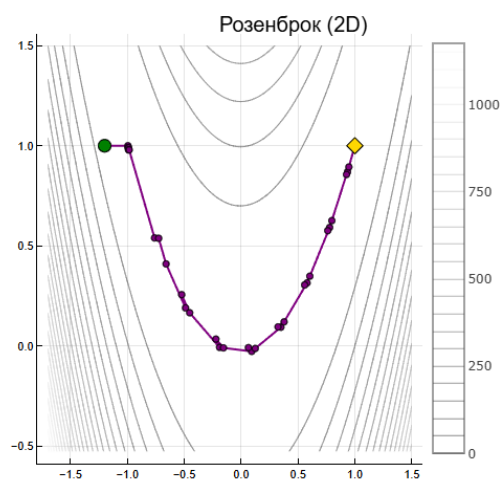


Рис. 1 — Розенброк

ФУНКЦИЯ: Растринг | Старт: [0.8, 0.8]
Метод Пауэлла: Итераций = 5, Найденный минимум = [1.99, 1.99]

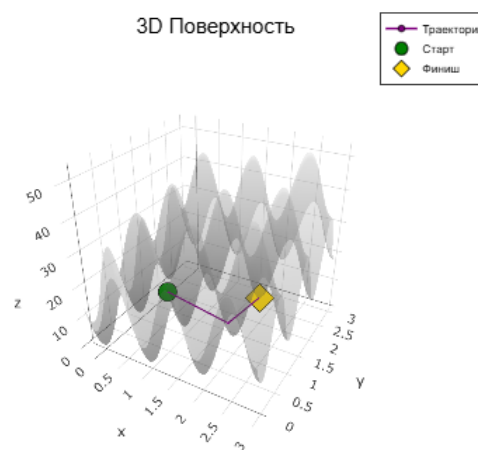
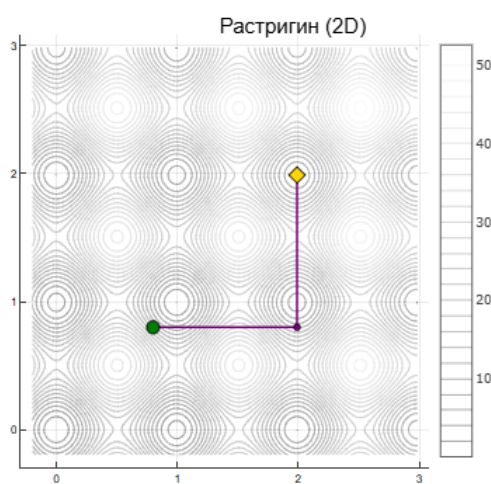


Рис. 2 — Растринг

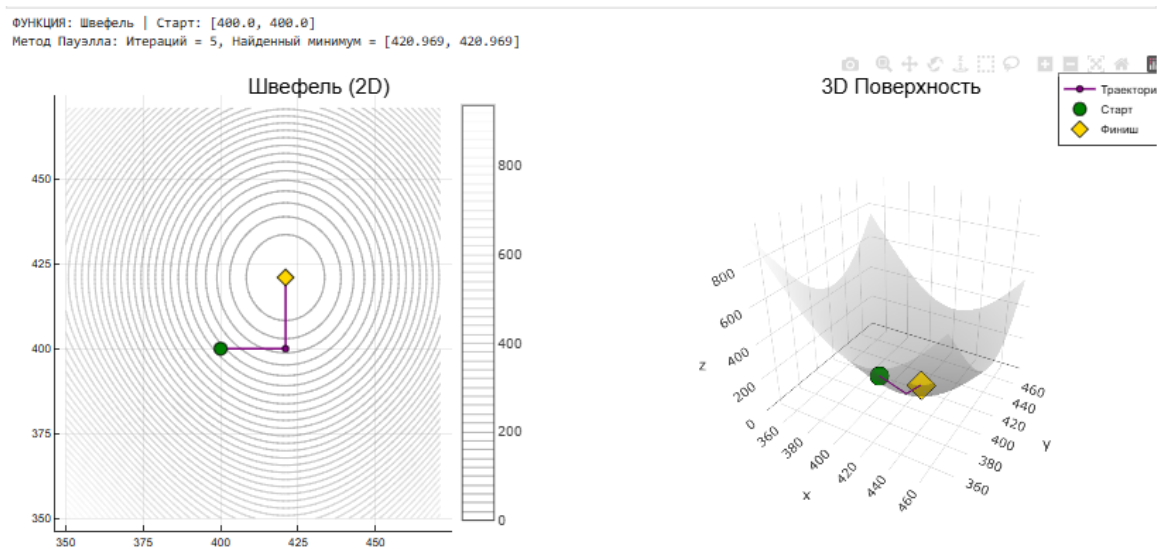


Рис. 3 — Швевель